Uvod u ROS

Branimir Ćaran

23.4.2025.| Regionalni centar izvrsnosti za robotske tehnologije

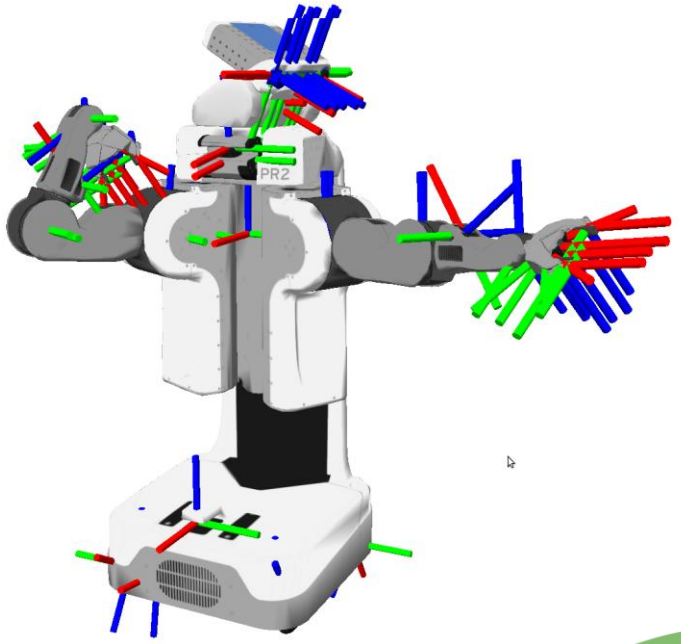
Plan predavanja

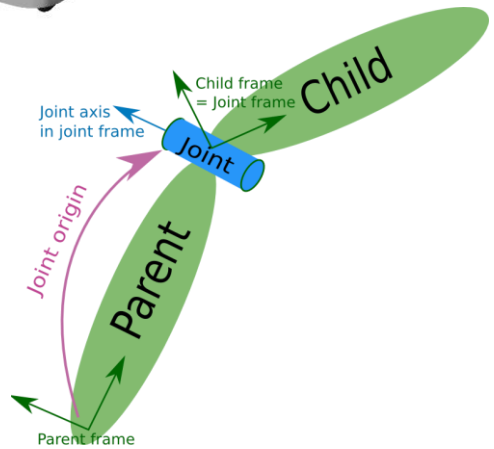
- *urdf*
- *urdf* – izrada osnovnih modela
- *Gazebo*
- *Gazebo* - primjeri

crta

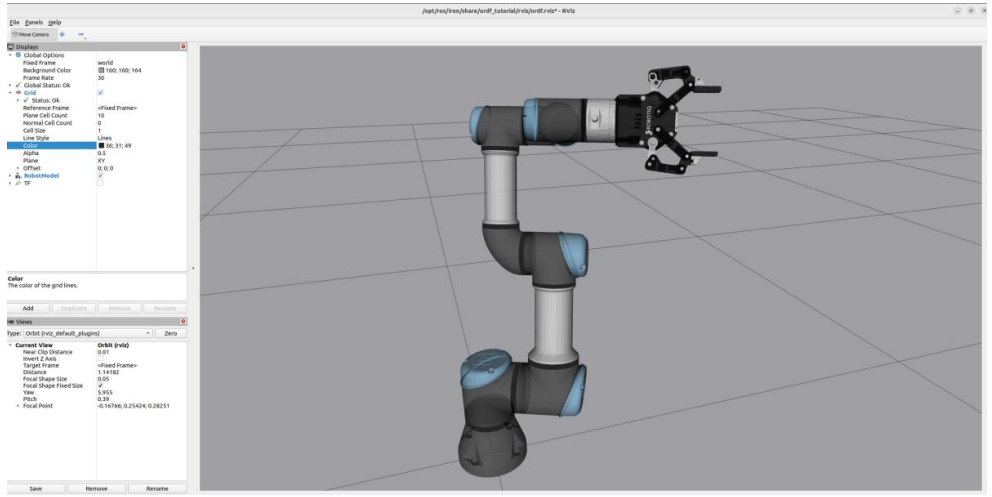
urdf

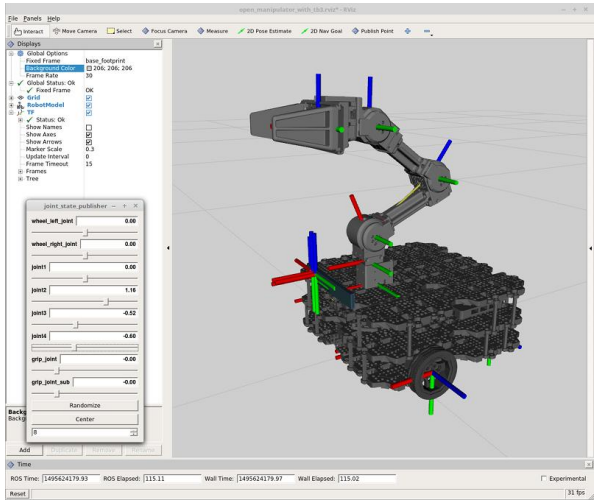
- *Universal Robot Description Format*
- XML datoteka koja se koristi za opis robota
- Definiraju se oblici, zglobovi i početni položaj
- Moguće je definirati i dinamičke parametre (masa, inercija, trenje)
- Koristi se *tf* za praćenje transformacija
- Moguće je definirati *.stl* ili *.dae* datoteku za pojedini link

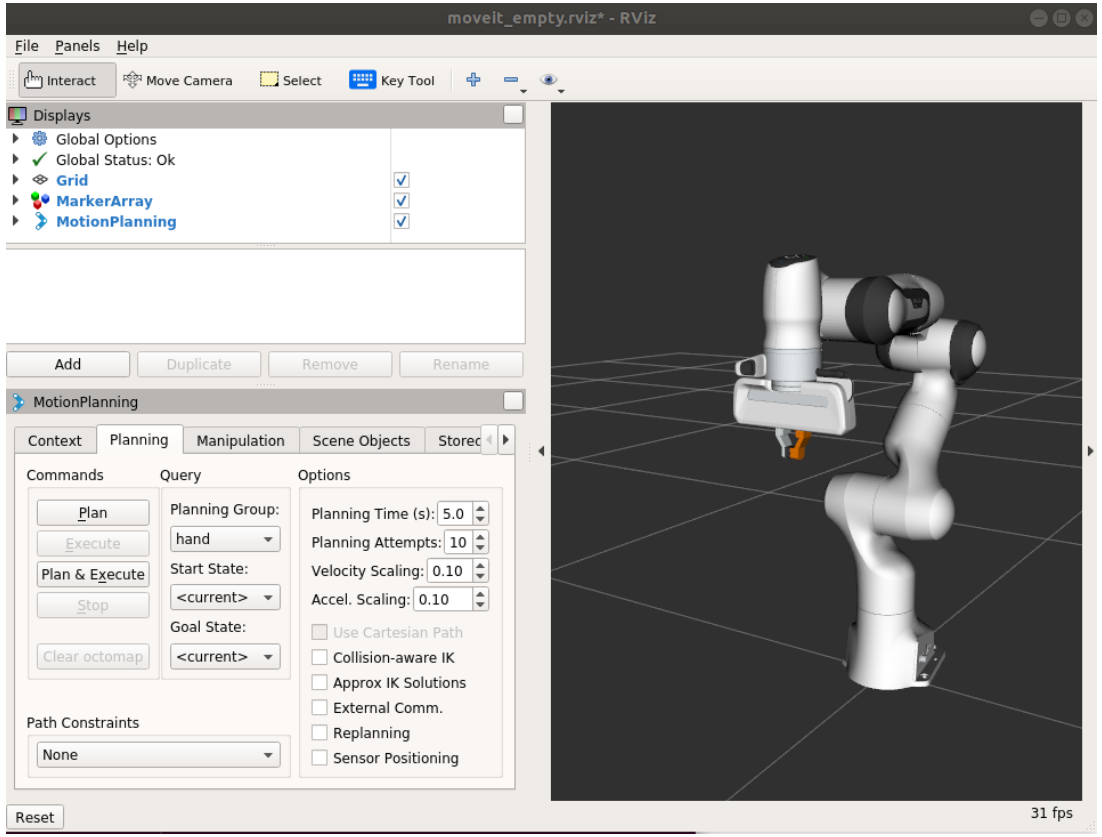




urdf

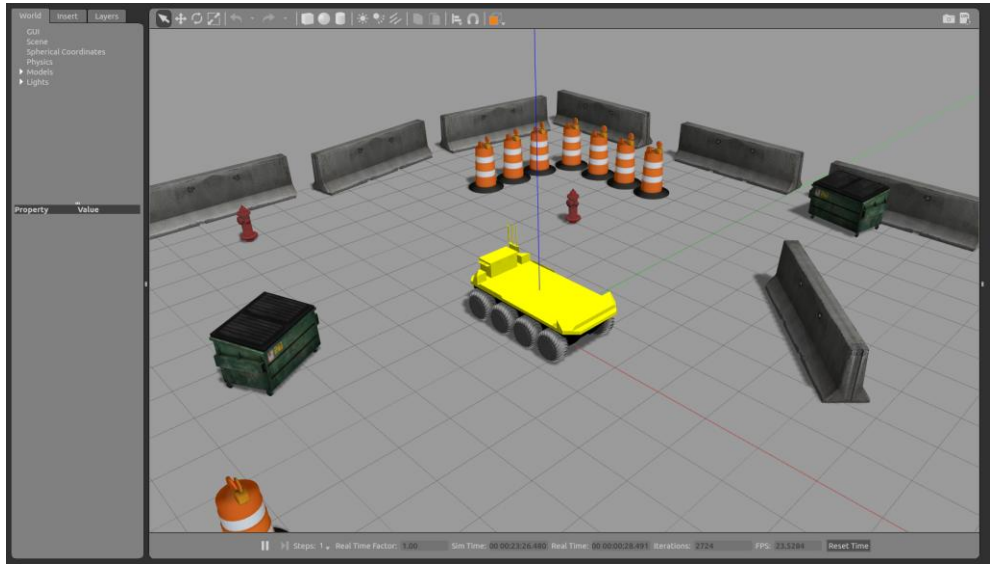





moveit_empty.rviz* - RViz
File Panels Help
Interact
Move Camera
Select
Key Tool
Displays
Global Options
Global Status: Ok
Grid
MarkerArray
MotionPlanning
Add
Duplicate
Remove
Rename
MotionPlanning
Context
Planning
Manipulation
Scene Objects
Commands
Query
Options
Plan
Execute
Plan & Execute
Stop
Clear octomap
Planning Group:
hand
Start State:
<current>
Goal State:
<current>
Planning Time (s): 5.0
Planning Attempts: 10
Velocity Scaling: 0.10
Accel. Scaling: 0.10
Use Cartesian Path
Collision-aware IK
Approx IK Solutions
External Comm.
Replanning
Sensor Positioning
Path Constraints
None
Reset
31 fps

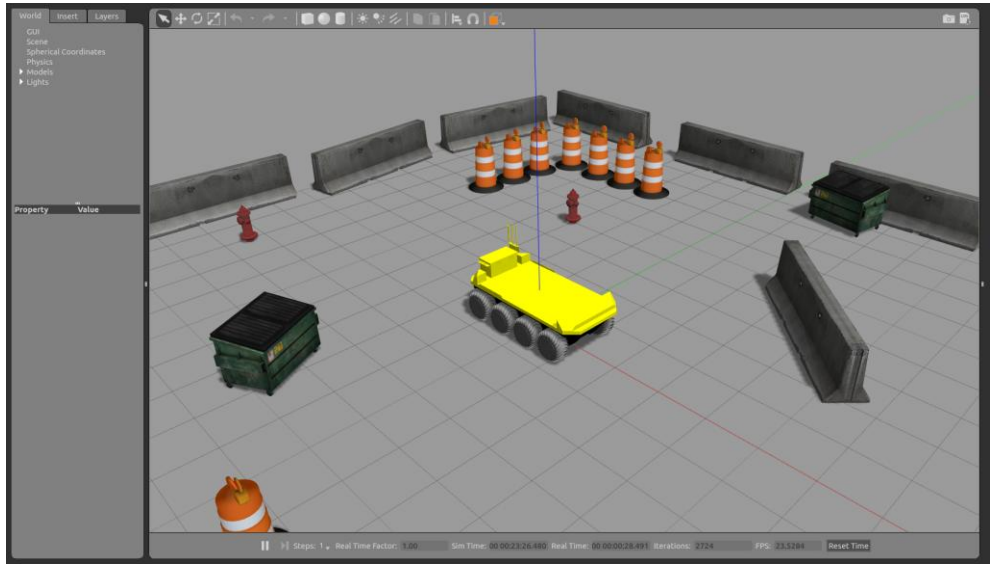
Gazebo

- Besplatni simulator
- Moguće je simulirati sve tipove robota i autonomnih sustava te njihovih podsustava
  - LIDAR senzor, kamere, GPS itd.
- Koristi nekoliko *physics engine*-a:
  - *Bullet, DAR, ODE, Simbody*
- Moguće je ostvariti komunikaciju s ROS2, MATLAB-om, Python, C++



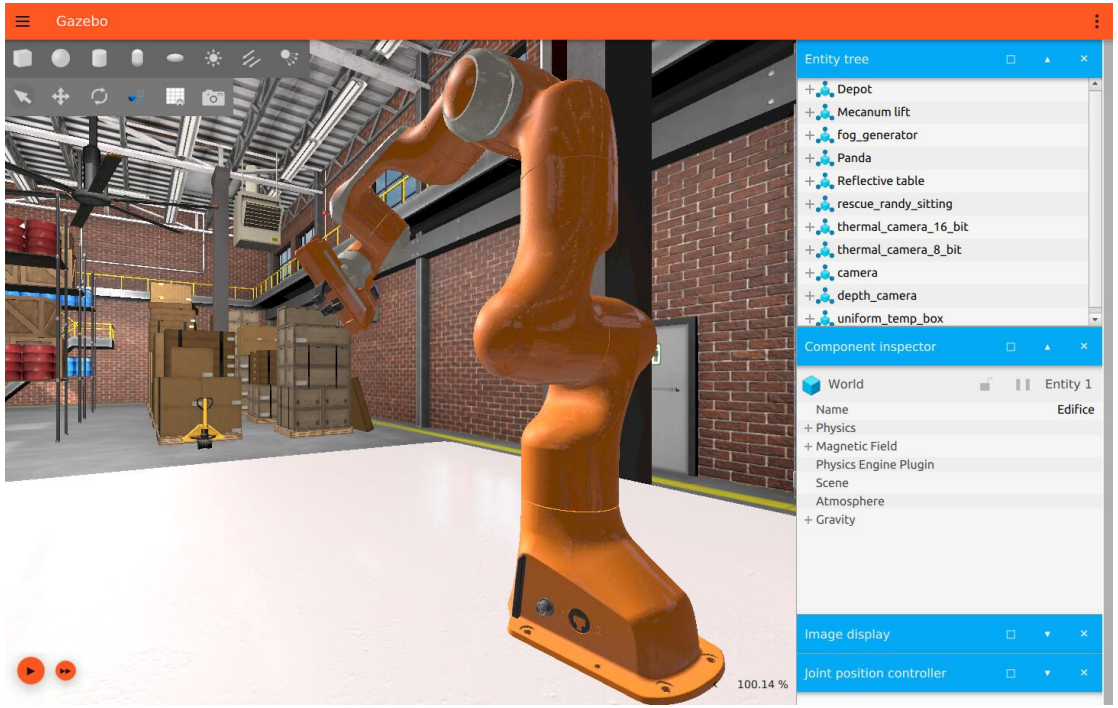
Gazebo

- Najbolji način za počet učiti robotiku
- Odličan za inicijalna testiranja robota i autonomnih sustava
- Jednostavan način za izradu digitalnog blizanca
- Testiranje algoritma u simulatoru i na stvarnom sustava
- Koristi *SDF File* (*simulation description format*)

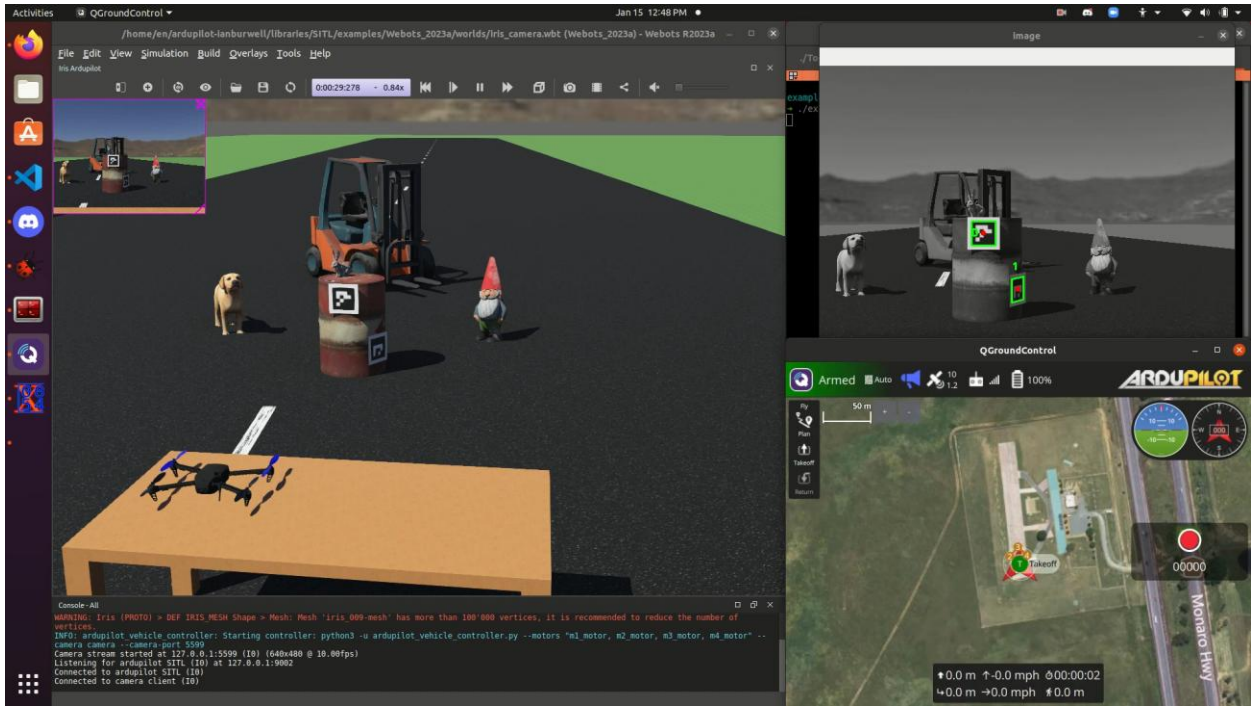


Gazebo - alternative

IGNITION

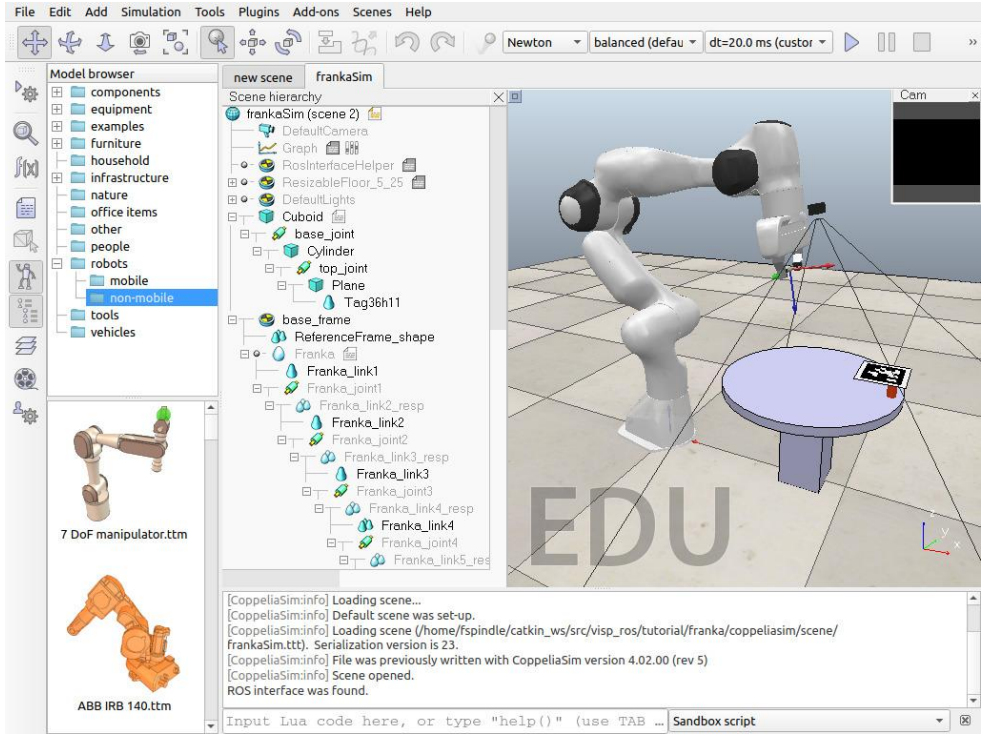


Webots™
robot simulation

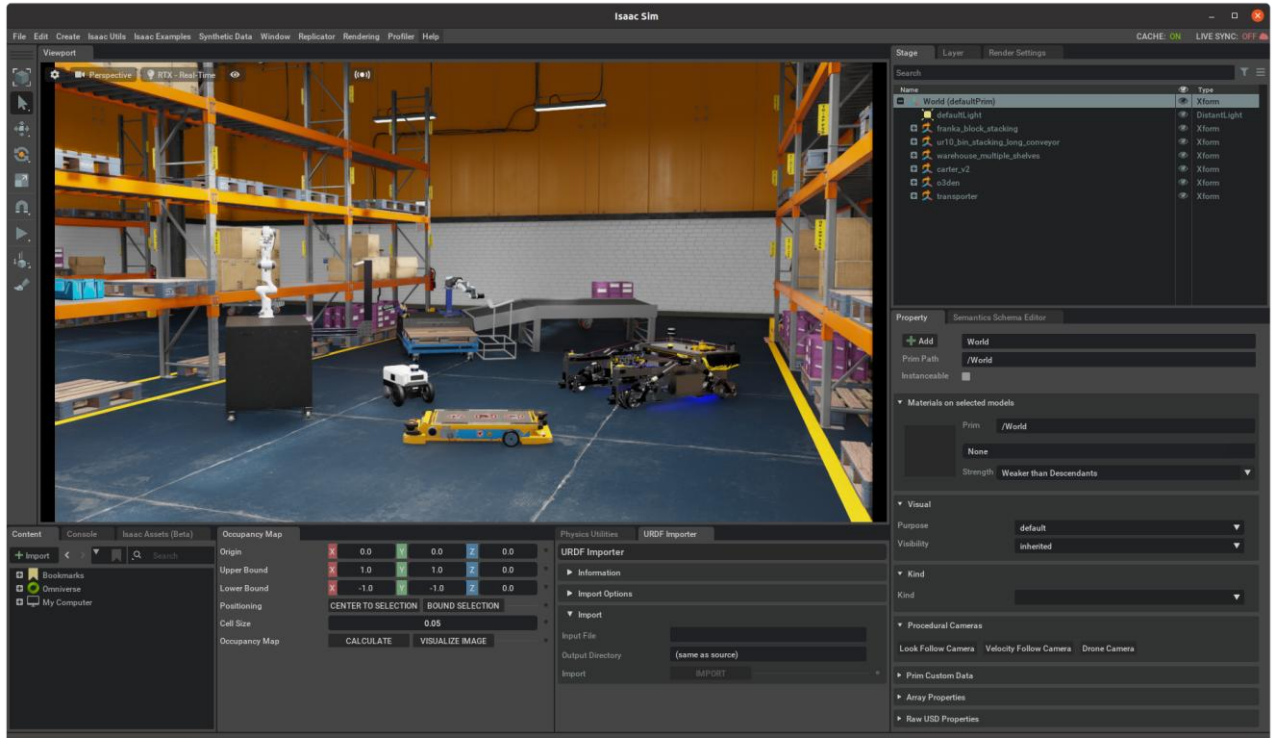


Gazebo - alternative

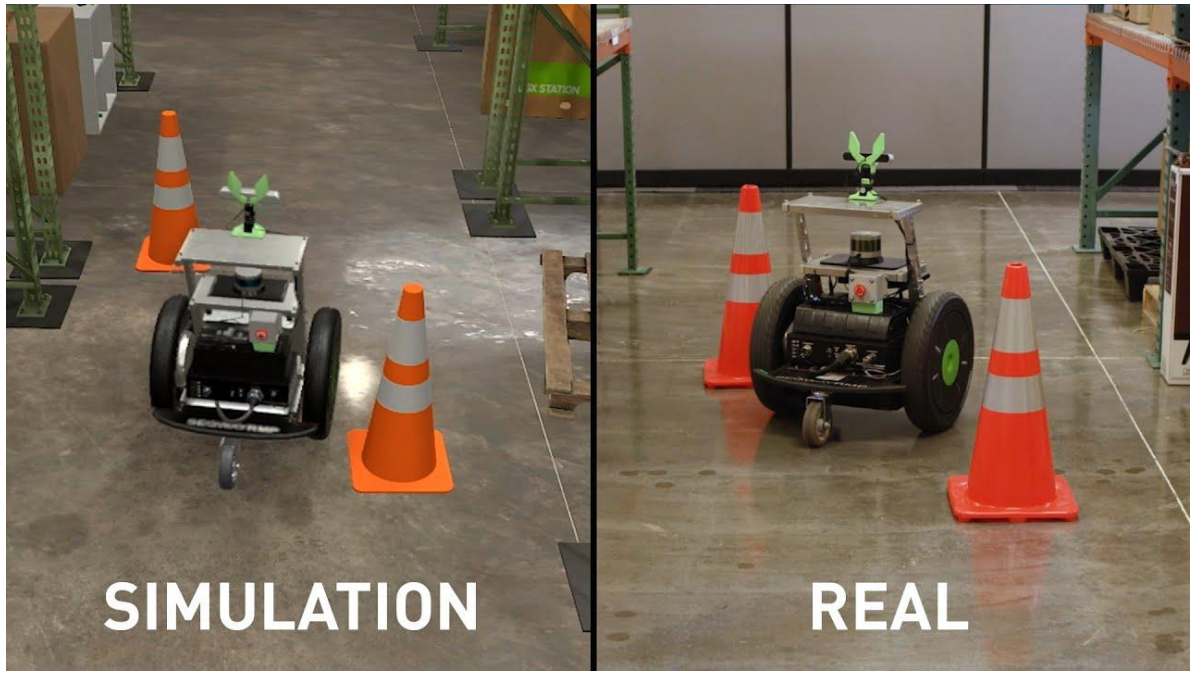
CoppeliaSim
from the creators of V-REP



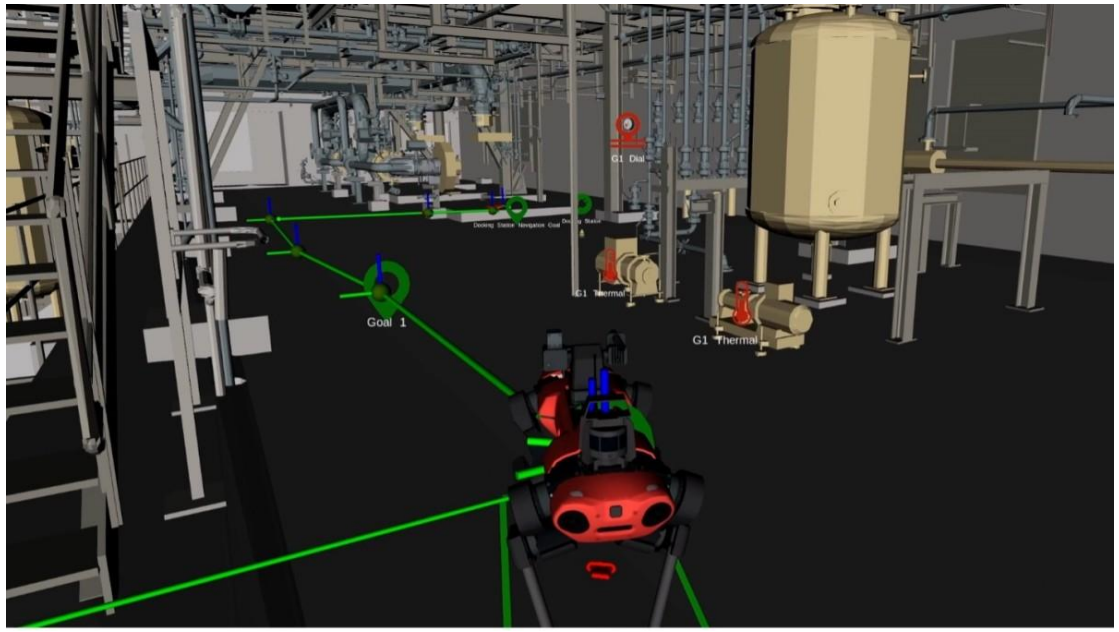
NVIDIA
ISAAC



Primjena simulatora



Primjena simulatora



Početak

- Napraviti radni direktorij, pa preuzeti paket i *build*-ati radni direktorij

```
mkdir -p vjezbe_urdf_ws/src
```

```
cd vjezbe_urdf_ws/src
```

```
git clone https://github.com/ros/urdf_tutorial.git -b ros2
```

```
cd ..
```

```
rosdep install -i --from-path src --rosdistro humble -y
```

```
colcon build
```

```
source install/setup.bash
```

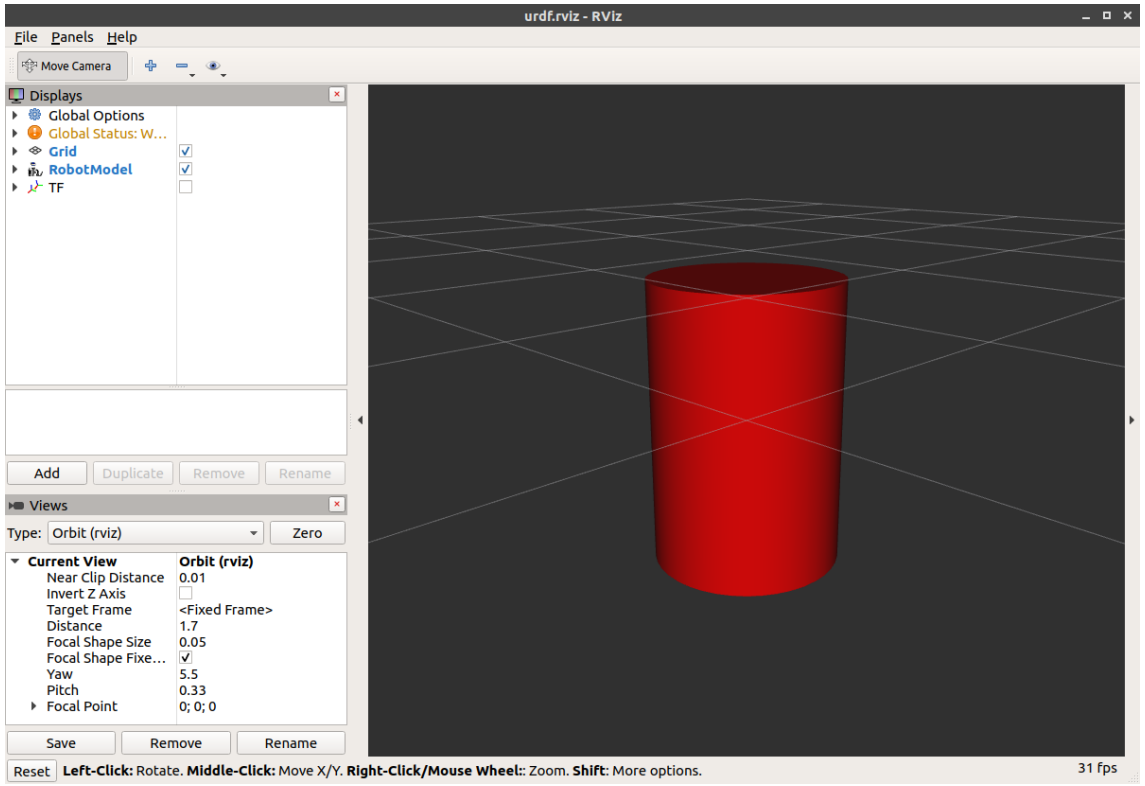
Pokretanje prvog primjera

- Potrebno je pokrenuti prvi primjer samo jednog oblika

```
ros2 launch urdf_tutorial display.launch.py model:=urdf/01-myfirst.urdf
```

- Pokreće se i sprema se model kasnije za *robot_state_publisher*
- Pokreće se čvor koji objavljuje na *joint_states*

```xml
<?xml version="1.0"?>
<robot name="myfirst">
  <link name="base_link">
    <visual>
      <geometry>
        <cylinder length="0.6" radius="0.2"/>
      </geometry>
    </visual>
  </link>
</robot>
```



Pokretanje drugog primjera

- Potrebno je pokrenuti drugi primjer s više oblika

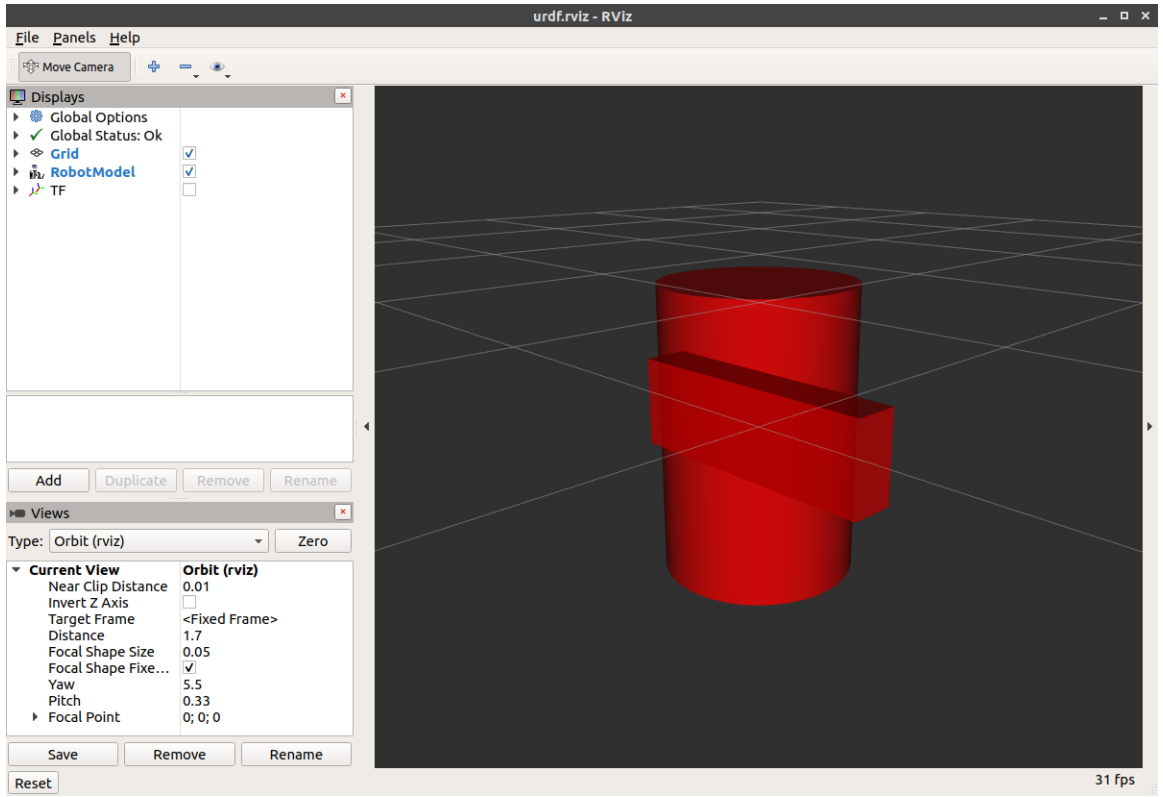
```
ros2 launch urdf_tutorial display.launch.py model:=urdf/02-multipleshapes.urdf
```

```xml
<?xml version="1.0"?>
<robot name="multipleshapes">
  <link name="base_link">
    <visual>
      <geometry>
        <cylinder length="0.6" radius="0.2"/>
      </geometry>
    </visual>
  </link>

  <link name="right_leg">
    <visual>
      <geometry>
        <box size="0.6 0.1 0.2"/>
      </geometry>
    </visual>
  </link>

  <joint name="base_to_right_leg" type="fixed">
    <parent link="base_link"/>
    <child link="right_leg"/>
  </joint>

</robot>
```



Pokretanje trećeg primjera

- Potrebno je pokrenuti treći primjer, s više oblika i definiranim ishodištima i transformacijama

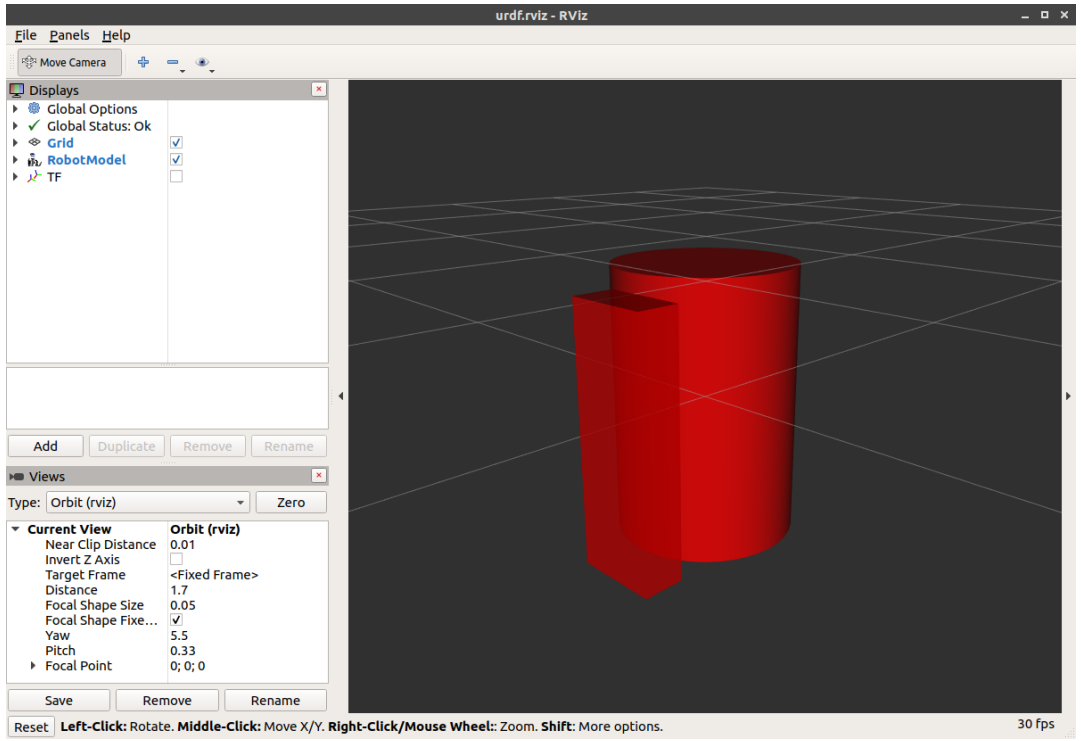
```
ros2 launch urdf_tutorial display.launch.py model:=urdf/03-origins.urdf
```

```xml
<?xml version="1.0"?>
<robot name="origins">
  <link name="base_link">
    <visual>
      <geometry>
        <cylinder length="0.6" radius="0.2"/>
      </geometry>
    </visual>
  </link>

  <link name="right_leg">
    <visual>
      <geometry>
        <box size="0.6 0.1 0.2"/>
      </geometry>
      <origin rpy="0 1.57075 0" xyz="0 0 -0.3"/>
    </visual>
  </link>

  <joint name="base_to_right_leg" type="fixed">
    <parent link="base_link"/>
    <child link="right_leg"/>
    <origin xyz="0 -0.22 0.25"/>
  </joint>

</robot>
```



Pokretanje četvrtog primjera

- Potrebno je pokrenuti četvrti primjer, s više oblika i definiranim „materijalom”

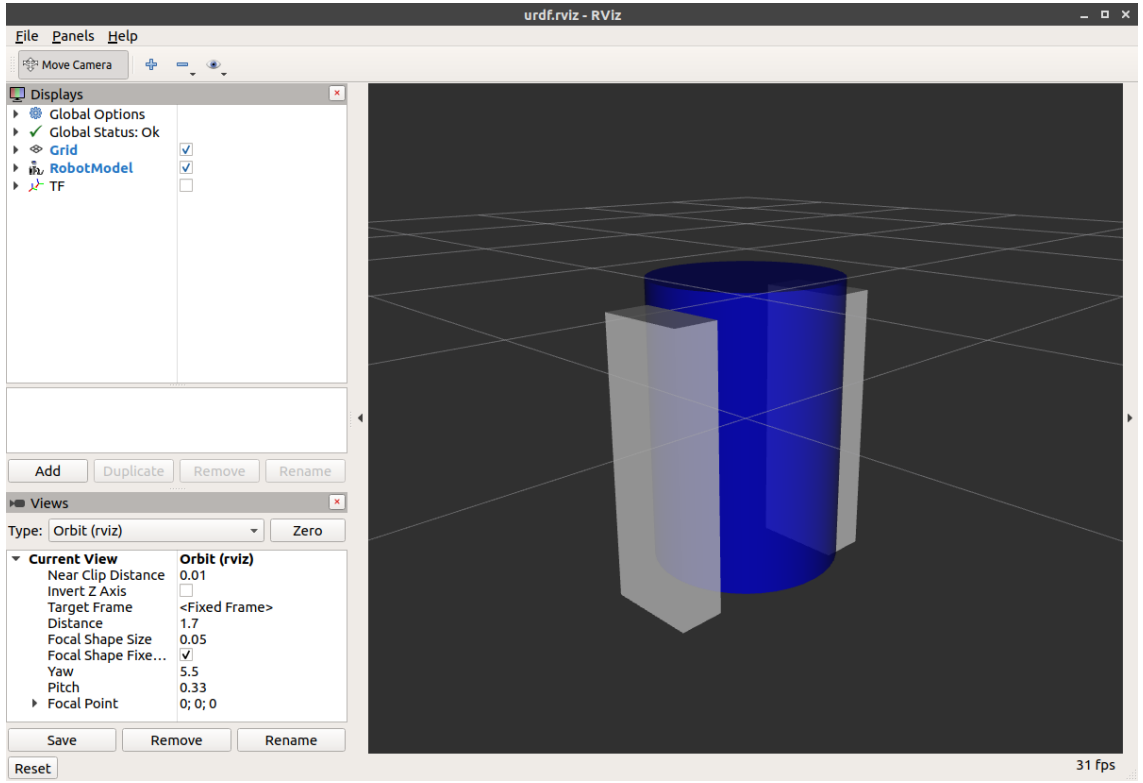
```
ros2 launch urdf_tutorial display.launch.py model:=urdf/04-materials.urdf
```

```xml
<?xml version="1.0"?>
<robot name="materials">

  <material name="blue">
    <color rgba="0 0 0.8 1"/>
  </material>

  <material name="white">
    <color rgba="1 1 1 1"/>
  </material>

  <link name="base_link">
    <visual>
      <geometry>
        <cylinder length="0.6" radius="0.2"/>
      </geometry>
      <material name="blue"/>
    </visual>
  </link>
```



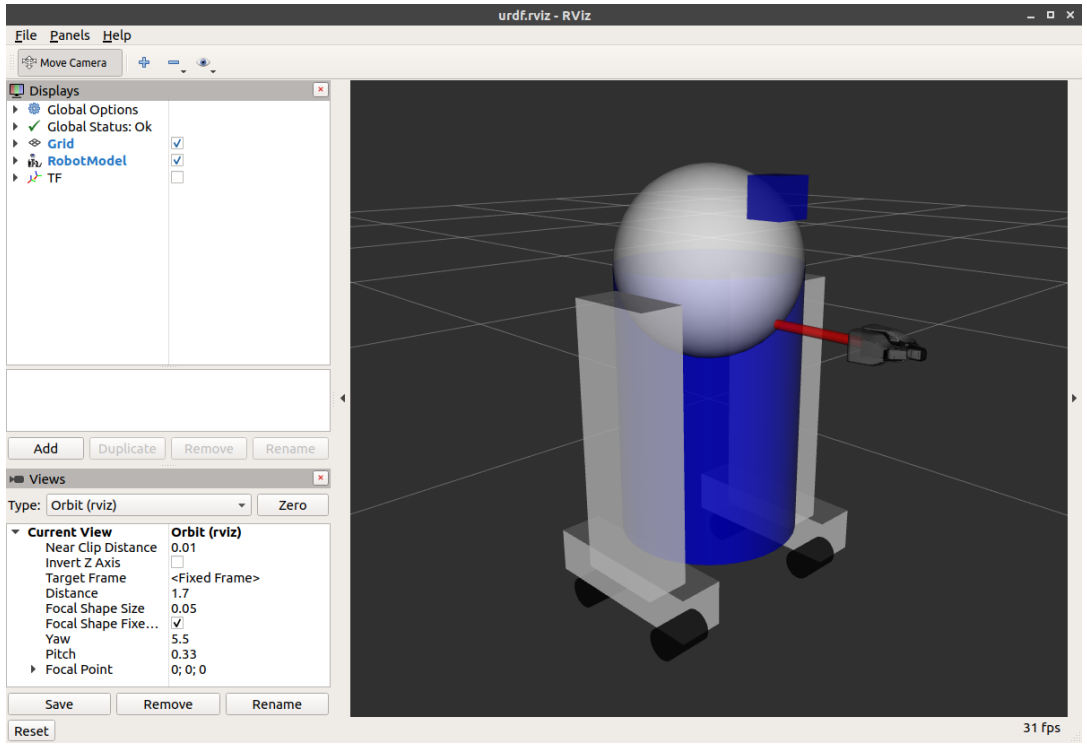
Pokretanje cijelog robota

- Potrebno je pokrenuti cijelog robota

```
ros2 launch urdf_tutorial display.launch.py model:=urdf/05-visual.urdf
```

```
<link name="head">
  <visual>
    <geometry>
      <sphere radius="0.2"/>
    </geometry>
    <material name="white"/>
  </visual>
</link>
```

```
<link name="left_gripper">
  <visual>
    <origin rpy="0.0 0 0" xyz="0 0 0"/>
    <geometry>
      <mesh filename="package://urdf_tutorial/meshes/l_finger.dae"/>
    </geometry>
  </visual>
</link>
```

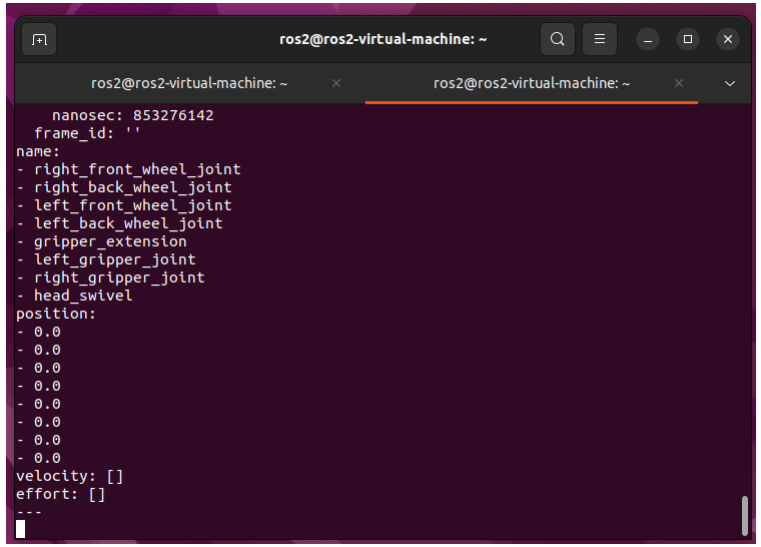


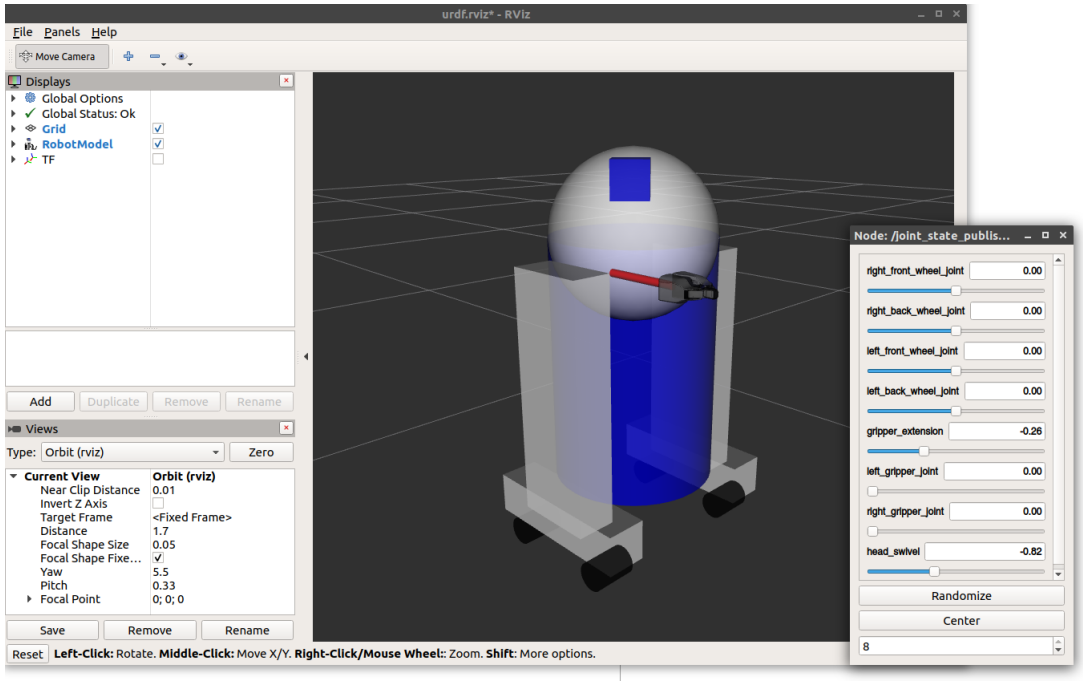
Pokretanje pokretnog robota

- Potrebno je pokrenuti šesti primjer, s pokretnim dijelovima

```
ros2 launch urdf_tutorial display.launch.py model:=urdf/06-flexible.urdf
```

```
ros2 topic echo /joint_states
```





Kontinuirani zglob

- Može se okretati od minus beskonačno do beskonačno
- Važno je definirati os oko koje se okreće

```
<joint name="head_swivel" type="continuous">
  <parent link="base_link"/>
  <child link="head"/>
  <axis xyz="0 0 1"/>
  <origin xyz="0 0 0.3"/>
</joint>
```

Revolutni zglob

- Sličan kao kontinuirani, ali postoje limiti
- Potrebno je definirati limite na poziciju, brzinu i moment(*engl. effort*)

```
<joint name="left_gripper_joint" type="revolute">
  <axis xyz="0 0 1"/>
  <limit effort="1000.0" lower="0.0" upper="0.548" velocity="0.5"/>
  <origin rpy="0 0 0" xyz="0.2 0.01 0"/>
  <parent link="gripper_pole"/>
  <child link="left_gripper"/>
</joint>
```

Prizmatični zglob

- Sličan kao kontinuirani, ali se pomiče translacijski
- Potrebno je definirati os translacije

```
<joint name="gripper_extension" type="prismatic">
  <parent link="base_link"/>
  <child link="gripper_pole"/>
  <limit effort="1000.0" lower="-0.38" upper="0" velocity="0.5"/>
  <origin rpy="0 0 0" xyz="0.19 0 0.2"/>
</joint>
```

Ostali tipovi zglobova

- Ravninski (*engl. planar*)
  - Može se pomicati bilo gdje u ravnini
- Plutajući (*engl. floating*)
  - Može se pomicati bilo gdje u prostoru

```
<joint name="gripper_extension" type="prismatic">
  <parent link="base_link"/>
  <child link="gripper_pole"/>
  <limit effort="1000.0" lower="-0.38" upper="0" velocity="0.5"/>
  <origin rpy="0 0 0" xyz="0.19 0 0.2"/>
</joint>
```

Kolizije

- Unutar *urdf* datoteke moguće se definirati kolizije
- Link na robota s ugrađenim kolizijama
- Uglavnom se geometrija za detekciju kolizije definira jednakom kao i vizualna geometrija osim kada
  - Želimo brže procesuiranje, pa se uvodi pojednostavljena geometrija
  - Želimo veću i robusniju sigurnosnu zonu oko robota

```
<link name="base_link">
    <visual>
      <geometry>
        <cylinder length="0.6" radius="0.2"/>
      </geometry>
      <material name="blue">
        <color rgba="0 0 .8 1"/>
      </material>
    </visual>
    <collision>
      <geometry>
        <cylinder length="0.6" radius="0.2"/>
      </geometry>
    </collision>
  </link>
```

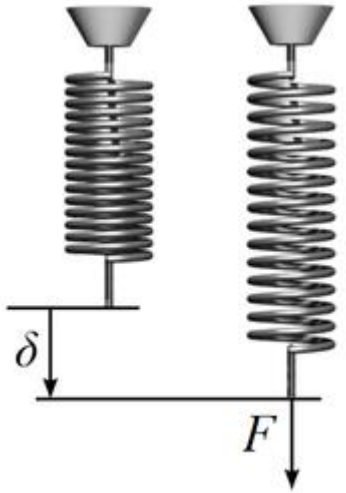
Inercija

- Masa je definirana u *kg*
- Matrica inercija je dimenzija 3x3
- $I = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{xy} & I_{yy} & I_{yz} \\ I_{xz} & I_{yz} & I_{zz} \end{bmatrix}$

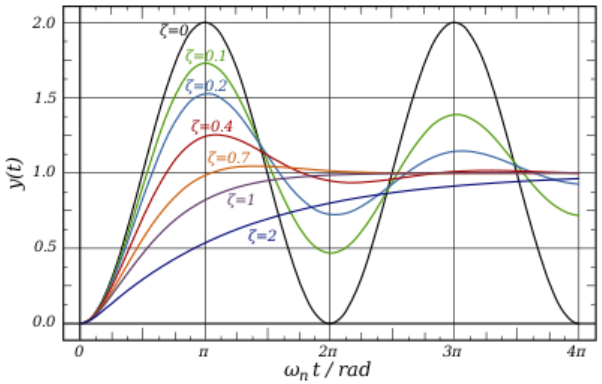
```
<link name="base_link">
  <visual>
    <geometry>
      <cylinder length="0.6" radius="0.2"/>
    </geometry>
    <material name="blue">
      <color rgba="0 0 .8 1"/>
    </material>
  </visual>
  <collision>
    <geometry>
      <cylinder length="0.6" radius="0.2"/>
    </geometry>
  </collision>
  <inertial>
    <mass value="10"/>
    <inertia ixx="1e-3" ixy="0.0" ixz="0.0" iyy="1e-3" iyz="0.0" izz="1e-3"/>
  </inertial>
</link>
```

Kontaktni parametri i dinamika zgloba

- Kontaktni parametri se podešavaju u *collision*-u
- Definiraju odnos između dva elementa u dodiru
  - Koeficijent trenja – *mu*
  - Koeficijent krutosti – *kp*
  - Koeficijent prigušenja – *kd*



- Dinamika zgloba se definira u *dynamics* tag-u za *joint*
  - Trenje – statičko trenje, za prizmatične zglobove u *N*, a revolutne u *Nm*
  - Prigušenje – dinamičko trenje, za prizmatične zglobove u *Ns/m,* a revolutne u *Nms/rad*



- Ostali *tag*-ovi koji se mogu koristiti u *urdf* datoteci su *calibration* i *safety controller*

xacro

- *xacro* je makro dodatak za *XML*
- *xacro* izvodi sve makro naredbe i daje izlazni rezultat
- *xacro* se najviše koristi za:
  - Konstante
  - Matematičke izraze
  - Makro naredbe
- Korištenje *xacro* datoteka

```
xacro model.xacro > model.urdf
```

- Pretvorba se može automatski napraviti u *launch* datoteci
- Potrebno je neko vrijeme da se generira

Primjena *xacro* u *launch* datoteci

- Potrebno je dodati *Command* kao parametar u *robot_state_publisher*-a

```
path_to_urdf = get_package_share_path('turtlebot3_description') / 'urdf' / 'turtlebot3_burger.urdf'
robot_state_publisher_node = launch_ros.actions.Node(
    package='robot_state_publisher',
    executable='robot_state_publisher',
    parameters=[{
        'robot_description': ParameterValue(
            Command(['xacro ', str(path_to_urdf)]), value_type=str
        )
    }]
)
```

Primjena *xacro* u *launch* datoteci

- Lakši način je koristiti paket *urdf_launch* koji automatski učitava *xacro/urdf*

```python
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.substitutions import PathJoinSubstitution
from launch_ros.substitutions import FindPackageShare


def generate_launch_description():
    ld = LaunchDescription()

    ld.add_action(IncludeLaunchDescription(
        PathJoinSubstitution([FindPackageShare('urdf_launch'), 'launch', 'display.launch.py']),
        launch_arguments={
            'urdf_package': 'turtlebot3_description',
            'urdf_package_path': PathJoinSubstitution(['urdf', 'turtlebot3_burger.urdf'])}.items()
    ))
    return ld
```

Preduvijet za korištenje *xacro*

- Na vrhu *urdf* datoteke potrebno je dodati *namespace*

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="firefighter">
```

Konstante i matematičke operacije u *xacro*

- Primjena konstanti

```
<link name="base_link">
  <visual>
    <geometry>
      <cylinder length="0.6" radius="0.2"/>
    </geometry>
    <material name="blue"/>
  </visual>
  <collision>
    <geometry>
      <cylinder length="0.6" radius="0.2"/>
    </geometry>
  </collision>
</link>
```

```
<xacro:property name="width" value="0.2" />
<xacro:property name="bodylen" value="0.6" />
<link name="base_link">
    <visual>
        <geometry>
            <cylinder radius="${width}" length="${bodylen}"/>
        </geometry>
        <material name="blue"/>
    </visual>
    <collision>
        <geometry>
            <cylinder radius="${width}" length="${bodylen}"/>
        </geometry>
    </collision>
</link>
```

- Primjena matematičkih operacija

```
<cylinder radius="${wheeldiam/2}" length="0.1"/>
<origin xyz="${reflect*(width+.02)} 0 0.25" />
```

Jednostavni makro

- Izrada jednostavnog makroa

```
<xacro:macro name="default_origin">
    <origin xyz="0 0 0" rpy="0 0 0"/>
</xacro:macro>
<xacro:default_origin />
```

- Generirati će

```
<origin rpy="0 0 0" xyz="0 0 0"/>
```

Parametrizirani makro

```
<xacro:macro name="default_inertial" params="mass">
    <inertial>
            <mass value="${mass}" />
            <inertia ixx="1e-3" ixy="0.0" ixz="0.0"
                 iyy="1e-3" iyz="0.0"
                 izz="1e-3" />
    </inertial>
</xacro:macro>
```

```
<xacro:macro name="blue_shape" params="name *shape">
    <link name="${name}">
        <visual>
            <geometry>
                <xacro:insert_block name="shape" />
            </geometry>
            <material name="blue"/>
        </visual>
        <collision>
            <geometry>
                <xacro:insert_block name="shape" />
            </geometry>
        </collision>
    </link>
</xacro:macro>

<xacro:blue_shape name="base_link">
    <cylinder radius=".42" length=".01" />
</xacro:blue_shape>
```

Primjena

```
ros2 launch urdf_tutorial display.launch.py model:=urdf/08-macroed.urdf.xacro
```

```
<xacro:macro name="leg" params="prefix reflect">
    <link name="${prefix}_leg">
        <visual>
            <geometry>
                <box size="${leglen} 0.1 0.2"/>
            </geometry>
            <origin xyz="0 0 -${leglen/2}" rpy="0 ${pi/2} 0"/>
            <material name="white"/>
        </visual>
        <collision>
            <geometry>
                <box size="${leglen} 0.1 0.2"/>
            </geometry>
            <origin xyz="0 0 -${leglen/2}" rpy="0 ${pi/2} 0"/>
        </collision>
        <xacro:default_inertial mass="10"/>
    </link>

    <joint name="base_to_${prefix}_leg" type="fixed">
        <parent link="base_link"/>
        <child link="${prefix}_leg"/>
        <origin xyz="0 ${reflect*(width+.02)} 0.25" />
    </joint>
    <!-- A bunch of stuff cut -->
</xacro:macro>
<xacro:leg prefix="right" reflect="1" />
<xacro:leg prefix="left" reflect="-1" />
```

Primjena *urdf*-a s *robot_state_publisher*-om

- Potrebno je napraviti novi radni direktorij *rsp_ws*

```
mkdir -p rsp_ws/src
```

```
cd src
```

- I u istom napraviti novi paket

```
ros2 pkg create --build-type ament_python --license Apache-2.0 rsp_tutorial --dependencies rclpy
```

- Napraviti folder *urdf* unutar paketa *rsp_tutorial*

```
cd rsp_tutorial
```

```
mkdir -p urdf
```

Primjena *urdf*-a s *robot_state_publisher*-om

- Potom otići u *urdf* folder i preuzeti *urdf* i *rviz* datoteke

```
cd urdf
```

```
wget https://docs.ros.org/en/humble/_downloads/872802005223ffdb75b1ab7b25ad445b/r2d2.urdf.xml
```

```
wget https://docs.ros.org/en/humble/_downloads/96d68aef72c4f27f32af5961ef48c475/r2d2.rviz
```

- Napraviti novi čvor i s e-učenja kopirati kod

```
cd rsp_tutorial/rsp_tutorial
```

```
touch state_publisher.py
```

```
code state_publisher.py
```

Primjena *urdf*-a s *robot_state_publisher*-om

- Napraviti *launch* folder unutar paketa *rsp_tutorial*

```
cd rsp_tutorial
```

```
mkdir -p launch
```

- Napraviti datoteku *demo.launch.py* i kopirati sadržaj s e-učenja

```
cd launch
```

```
touch demo.launch.py
```

```
code demo.launch.py
```

Primjena *urdf*-a s *robot_state_publisher*-om

- Urediti *setup.py*

```
import os
from glob import glob
from setuptools import setup
from setuptools import find_packages
```

```
data_files=[
  ...
  (os.path.join('share', package_name, 'launch'), glob(os.path.join('launch',
'*launch.[pxy][yma]*'))),
  (os.path.join('share', package_name), glob('urdf/*')),
],
```

```
'console_scripts': [
    'state_publisher = rsp_tutorial.state_publisher:main'
],
```

Primjena *urdf*-a s *robot_state_publisher*-om

- Urediti *build* i pokretanje

```
cd rsp_ws
```

```
colcon build
```

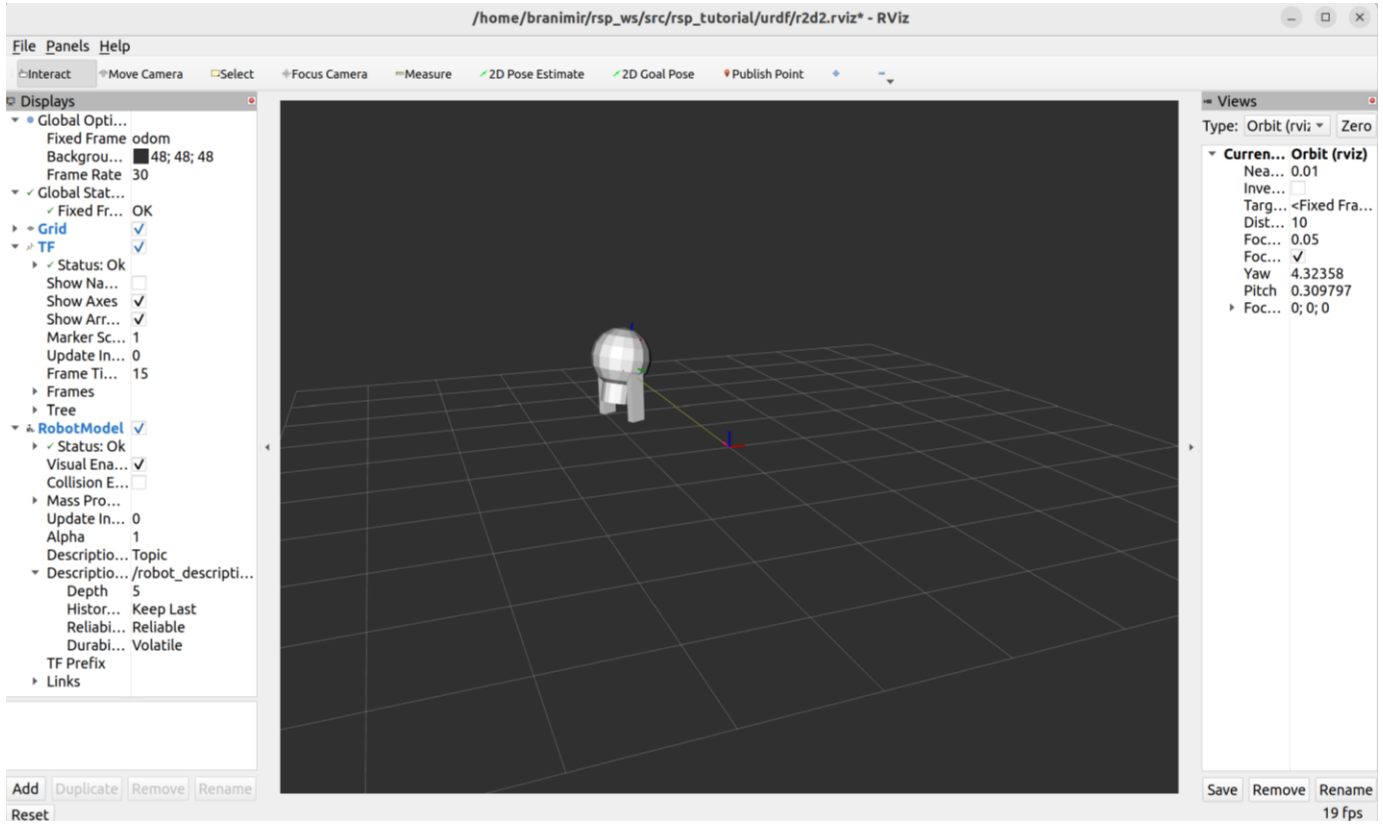
```
source install/setup.bash
```

```
ros2 launch rsp_tutorial demo.launch.py
```

```
rviz2
```

Primjena *urdf*-a s *robot_state_publisher*-om

- Učitati *rviz* konfiguracijsku datoteku *File -> Open Config*



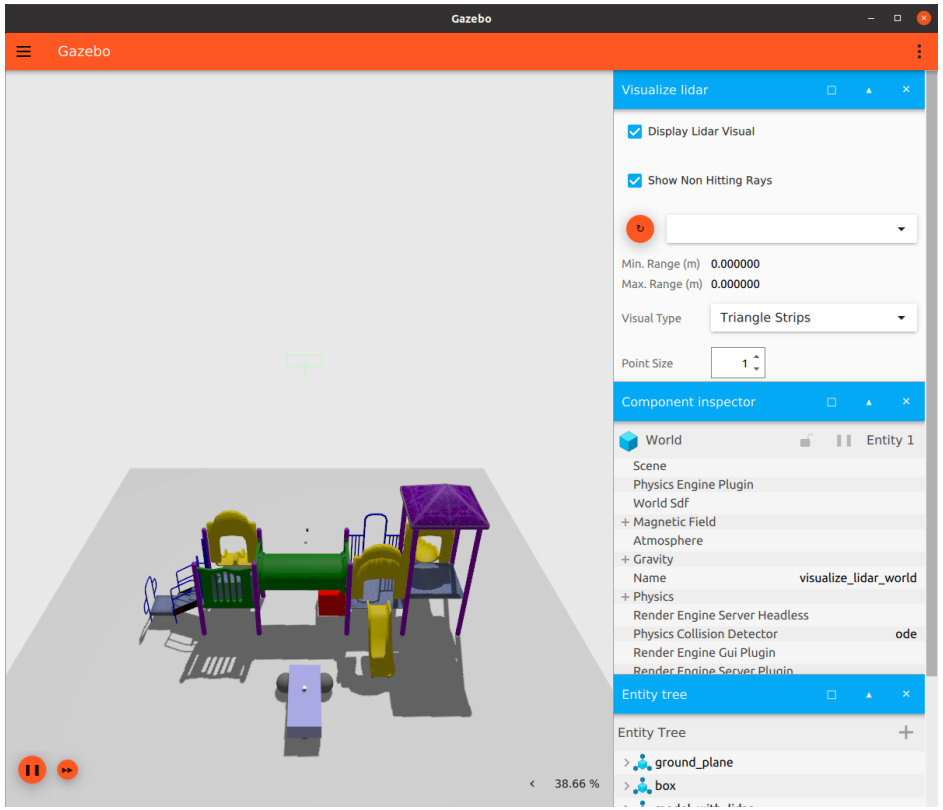
Instalacija Gazebo simulatora

```
sudo apt-get install ros-${ROS_DISTRO}-ros-gz
```

```
sudo apt-get install ros-humble-ros-ign-bridge
```

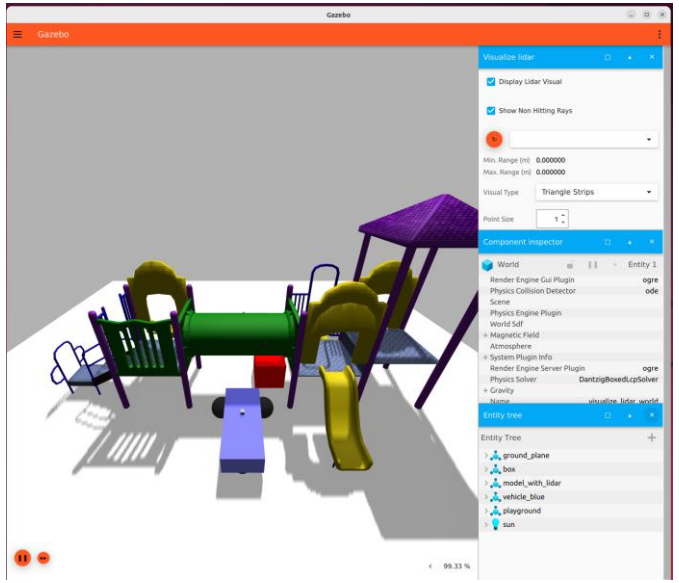
- Provjera instalacije

```
ign gazebo -v 4 -r visualize_lidar.sdf –render-engine ogre
```



Instalacija Gazebo simulatora

- Povezivanje Gazebo simulatora i ROS2

```
ros2 run ros_gz_bridge parameter_bridge
/model/vehicle_blue/cmd_vel@geometry_msgs/msg/Twist]ignition.msgs.Twist
```

```
ros2 topic pub /model/vehicle_blue/cmd_vel geometry_msgs/Twist "linear: { x: 0.1 }"
```